面向对象：

obj:

眼睛，

头发，

手臂，

耳朵；

.....

```
var person = {
        eye:"big",
        hair:"long",
        leg:"long",
        money:"rich",
        skin:"white"
         smoking:function(){
            console.log( this )
            this.health--
        },
        drink:function(){
            this.health++
        }
}
//添加属性：obj.attr =  value;
//删除属性：delete obj.attr ;
//改变属性：obj.attr ; => 把属性给覆盖;
//查：  obj.attr;
```

构造函数

```
//{name: "梅赛德斯奔驰 S400", year: "2020", health: "90%", run: "2W", color: "black"}

//构造函数;
function Benz(){

    // var name = "梅赛德斯奔驰 S400";
    // var year = "2020";
    // console.log( this )
    this.name = "梅赛德斯奔驰 S400";
    this.year = "2020";
    this.health = '90%';
    this.run = "2W";
    this.color = "black";

}

// Benz()

//实例化对象:
// console.log( new Benz() )
//new Array();
var car1 = new Benz();
var car2 = new Benz();
var car3 = new Benz();
var car4 = new Benz();
```

可以传参

```
//{name: "梅赛德斯奔驰 S400", year: "2020", health: "90%", run: "2W", color: "black"}

//构造函数;
function Benz(name,year){

    // var name = "梅赛德斯奔驰 S400";
    // var year = "2020";
    // console.log( this )
    // this.i = i
    this.name = name;
    this.year = year;
    this.health = '90%';
    this.run = "2W";
    this.color = "black";
    console.log( this )   //指向每一个实例对象;

}

// Benz()

//实例化对象:
// console.log( new Benz() )
//new Array();

var car1 = new Benz("大鸡",2020);
var car2 = new Benz("SL",2020);
var car3 = new Benz("E",2020);
var car4 = new Benz("C",2020);
```

面向对象的原理

```
17
18      function Benz(name,year){
19
20          //{name: "梅赛德斯奔驰 S400", year: "2020", health: "90%", run: "2W", color: "black"}
21          // var name = "梅赛德斯奔驰 S400";
22          // var year = "2020";
23          // console.log( this )
24          // this.i = i
25          var _this = {}
26          _this.name = name;
27          _this.year = year;
28          _this.health = '90%';
29          _this.run = "2W";
30          _this.color = "black";
31          // console.log( this )   //指向每一个实例对象;
32
33
34          console.log( _this )
35
36          return _this;
37
38      }
39
40
41      var car1 = Benz("梅赛德斯奔驰 S400",2018)
42      console.log( car1 )
43
44
45
```

DevTools - 127.0.0.1:5500/04%20%E9%9D%A2%E5%90%91%E5%AF%B9%E8%B1%A1%E7%9A%84%E5%8E%9F%E7%90%86.html

Elements Performance Console Sources Network Memory Application Security Lighthouse

top Filter Default levels

```
{name: "梅赛德斯奔驰 S400", year: 2018, health: "90%", run: "2W", color: "black"}   04_面向对象的原理.html:34
{name: "梅赛德斯奔驰 S400", year: 2018, health: "90%", run: "2W", color: "black"}   04_面向对象的原理.html:42
>
```

面向对象的三大特征：

封装、继承、多态【多种形态】

链式调用

```
25      var animo = {
26          eat:function( type ){
27              if( type === "cat" ){
28                  console.log( '猫粮' )
29              }else if( type === 'dog' ){
30                  console.log( '狗粮' )
31              }else{
32                  console.log( '米饭' )
33              }
34          },
35          drink:function( type ){
36              switch( type ){
37                  case 'water':
38                      console.log( "正常" )
39                      return {
40                          name:function(){
41                              console.log( "名字" )
42                          }
43                      }
44                      break;
45                  case "milk":
46                      console.log( "幼崽" )
47                      break;
48                  default:
49                      console.log( "喝jio" )
50              }
51          },
52          sleep:"oneday",
53          run(){
54          }
55      }
56      console.log( animo.drink("water").name() )
57      //$('box').css({width:100,height:100}).animation({width:200,height:200}).ajax() ...
```

DevTools - 127.0.0.1:5500/05%20%E5%A4%9A%E6%80%81.html

Elements Performance Console Sources Network Memory Application Security Lighthouse

top Filter Default levels

```
正常                    05_多态.html:38
名字                    05_多态.html:41
undefined               05_多态.html:56
>
```

__proto__　公共的仓库

```
    Person.prototype.sex = '男';
    Person.prototype.house = true;
    function Person(name,tall,weight){
        this.name=name,
        this.tall=tall,
        this.weight=weight;
    }
    var xiaoming = new Person('xiaoming','160','106');
    var xiaopan = new Person('xiaopan','180','160');
    var xiaogou = new Person('xiaogou','180','120');
    //构造函数中没有的值，却能拿到，是因为通过 __proto__'公共仓库拿到的'

```

删除、修改原型、构造器

```
    //这样更改不会影响到原型(公共的仓库)的值：
    xiaoming.sex = '女';
    xiaopan.house = false;
    console.log(xiaoming)
    console.log(xiaopan)

    //删除原型的键值对：
    delete Person.prototype.house
    //修改原型的键值对：
    Person.prototype.sex = '女';

    //每个人的公共的小仓库都是相等的，因为都引用了公共的仓库：
    console.log(xiaoming.__proto__ === xiaopan.__proto__)//true

    //改变其中一个人的__proto__，另外的__proto__都会被改变，因为它们都引用自同一个公共仓库
    xiaopan.__proto__.sex = 'XX'



    //constructor :构造器【构造函数】

    //同一个构造器它们的constructor相等
    console.log(xiaoming.constructor===xiaopan.constructor)//true
    // console.log(xiaogou.constructor)
    // console.log(xiaopan.constructor)

    function Person1(){}
    var per1 = new Person1()
    //不同的构造器它们的constructor不等
    console.log(per1.constructor===xiaoming.constructor)//false
</script>

</html>
```

写在一块的效果一样

```
        //写在一起作用一样:
        Person.prototype = {
            sex: '男',
            house: true
        }


        function Person(name, tall, weight) {
            this.name = name,
                this.tall = tall,
                this.weight = weight;
        }
        var xiaoming = new Person('xiaoming', '160', '106');
        var xiaopan = new Person('xiaopan', '180', '160');
        var xiaogou = new Person('xiaogou', '180', '120');
    </script>
```

本身有值先取本身

有多个取到的是实例化之前设定的值

```
        Person.prototype.name = 'xiao'
        function Person(){
            // this.name = 'summy'  //如果本身有值就直接取本身的
        }
        Person.prototype.name = 'yue'

        Person.prototype ={
            name:'QWER'
        }
        var per1 = new Person();
        var per2 = new Person();

        //取到的是实例化之前设定的值
        console.log( per2 )
        console.log( per2.name )//QWER
```

通过把实例化赋值给原型：就形成了原型链

形成链的各个实例化可以互相拿到原本自己没有的值

```
Suiyue.prototype.Gf = true
function Suiyue(){
    this.xin = '8K'
}
var suiyue = new Suiyue()


Bashi.prototype.Car = 'LSLS'
function Bashi(){
    this.xin = "20K"
}
var bashi = new Bashi()


Xiaofang.prototype.house = true
Xiaofang.prototype = bashi;
function Xiaofang(){
    this.xin = "40K"
}
var xiaofang = new Xiaofang()


Summy.prototype.money = "-100W"
Summy.prototype.xin = '20W'
Summy.prototype = xiaofang;
function Summy(){
    this.xin = "0.2W"
}
var summy = new Summy()


console.log(summy.xin)
console.log(summy)
```

call改变原例的指向

```
Person.prototype.abc = 'abc'
function Person( name , tall , weight ){

    this.name = name;
    this.tall = tall;
    this.weight = weight;

}

var obj = {}
Person.call( obj , 'summy',"179" , "172" )

var per1 = new Person( 'shaonian',"180" , "150" );

console.log( per1 )

console.log( obj )//obj里面就不会有原型里的abc，通过call来改变原型的指向避免了共享

```

两个对象实习共享

```
Summy.prototype.house = false;
Bashi.prototype.money = true;
function Summy(){
    this.name = "666"
}
function Bashi(){
    this.name = "789"
}

//通过赋值原型，两个对象实现了共享;
Summy.prototype = Bashi.prototype;

var Sun1 = new Summy()
var ba = new Bashi();
Summy.prototype.niu = '3'
Bashi.prototype.eat = true;

```

实现单方面访问

调用封装的函数，必须要在实例化之前调用，否则没用。

```
13        function Summy() {
14            this.name = "666"
15        }
16        Summy.prototype.niu = 100;
17        Summy.prototype.car = true;
18
19        function Bashi() {
20            this.name = "789"
21        }
22        Bashi.prototype.money = 1000;
23
24        //两个参数（供给，接收）
25        //单方面共享;
26        //1.构造一个空函数
27        //2.（供给）的公共赋值给空函数的公共
28        //3.实例化空函数赋值给（接收）的公共
29        //4.(接收)赋值给接收公共的constructor.
30        //供给     接收
31        //第二个值可以访问第一个，第一个值不能访问第二个
32        function inheritAttr(target, Origin) {
33            function F(){};
34            F.prototype = target.prototype;
35            Origin.prototype = new F();
36            Origin.prototype.constructor = Origin;
37        }
38
39        inheritAttr(Summy,Bashi );
40
41        var summy = new Summy()
42        var bashi = new Bashi()
43
44
45
46
47        console.log(summy)
48        console.log(bashi)
```

instanceof 操作符;

```
// instanceof 操作符;


function Person(){}
function Str(){}

var per1 = new Person();
var per2 = new Person();
var per3 = new Person();
var str1 = new Str();


console.log(  per1 instanceof Person )  //t s per1是否为Person的一个实例;
console.log(per1 instanceof Str)//false per1是否为Str的一个实例;
console.log( str1 instanceof Person ) //false

// 判断某个对象是否为XXX类的实例;
console.log( [] instanceof Object ) //t
console.log( [] instanceof Array ) //t
console.log( Array instanceof Object ) //t
console.log( Function instanceof Object ) //t
console.log( String instanceof Object ) //t
```

判断[]　{}

1. 判断长度;

   数组有长度，json没有长度;

2. 判断是否instranceOf Array;

3. 判断两者的构造器;

4. Object.prototype.toString.call([])

   [object Array] => [object Object]

```
console.log(Object.prototype.toString.call([]))//[object Array]
console.log(Object.prototype.toString.call({}))//[object Object]
```

可枚举hasOwnProperty

```
21      var obj = {
22          name:"少年",
23          age:23,
24          tall:180,
25          weight:80,
26          __proto__:{   //__proto__ 它是不可枚举的
27              money:6666
28          }
29      }
30      /*
31          枚举与不可枚举:
32              枚举：深紫色，可以遍历到;
33              不可枚举：浅紫色的，不可遍历到的;
34      */
35
36      console.log( obj )
37      for( var key in obj ){
38          console.log( key )//返回里面的键(属性)
39
40          //hasOwnProperty:可枚举;   看是否可以枚举返回布尔值
41          console.log( obj.hasOwnProperty( key ) )
42
43
44      }
45
```